30-DAY AI & HYBRID DEEP LEARNING SPRINT • PHASE 1: CLASSICAL ML MECHANICS

Day 04: PCA Image Compression & Denoising

Singular Value Decomposition (SVD), Eigenface Subspaces, and Low-Rank Noise Filtering

1. Why Compress & Denoise Pixels with Linear Subspaces?

Core Intuition

A 64×64 grayscale photograph contains $D = 4{,}096$ pixels, yet natural images do not fill this 4,096-dimensional space randomly. Adjacent cheek or forehead pixels are strongly correlated, meaning real faces lie on a low-dimensional linear subspace. Principal Component Analysis (PCA) rotates the coordinate axes to align with directions of maximum variance. By retaining only the top k axes (Eigenfaces), we compress storage by 80× and strip away unaligned random camera noise.

2. SVD Factorization & Subspace Reconstruction

Mathematical Engine

STEP A • MEAN CENTERING & ECONOMY SVD

$$X_c = X - \mu = U \Sigma V^T \quad | \quad \lambda_i = \sigma_i^2 / (N - 1)$$

Instead of forming the huge 4096×4096 covariance matrix $C = X_c^T X_c/(N-1)$, SVD extracts orthonormal eigenvectors V and variance λ_i directly.

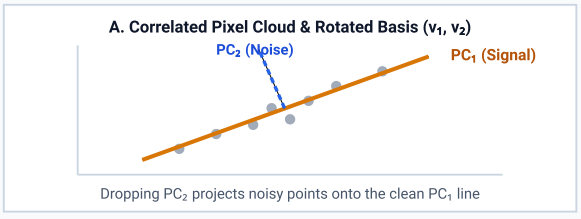
STEP B • BOTTLENECK PROJECTION & RECONSTRUCTION

$$Z_k = (X - \mu)\, V_k \quad | \quad \hat{X}_k = Z_k\, V_k^T + \mu$$

Projects D pixels into k latent coordinates Z_k ($k \ll D$). Reconstructing $\hat{X}_k$ discards orthogonal high-frequency noise in tail components.

3. Geometric Mechanics: Orthogonal Rotation & Spectral Noise Cutoff

Visual Topology



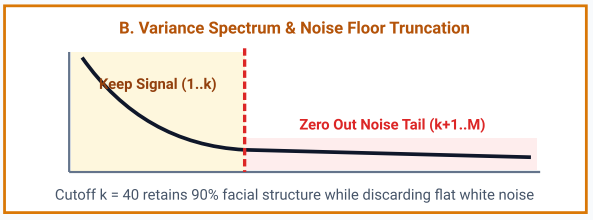


Figure 4.1: (A) Orthogonal rotation to principal axes of variance; (B) Spectral truncation separating structured facial signal from isotropic noise.

4. SVD-PCA Tensor Shapes & Memory Compression Reference

NumPy Architecture

When working with $N = 400$ images of $64 \times 64 = 4{,}096$ pixels ($N < D$), economy SVD (`full_matrices=False`) avoids allocating a massive 4096×4096 matrix and caps rank at $M = 400$.

PIPELINE STAGE	MATHEMATICAL EXPRESSION	VECTORIZED NUMPY CODE	TENSOR SHAPE
1. Flatten & Center	$X_c = X - \mu$	`Xc = X - X.mean(axis=0)`	`(400, 4096)` float32
2. Economy SVD	$U, \Sigma, V^T = \text{svd}(X_c)$	`U, S, Vt = np.linalg.svd(Xc, False)`	`Vt: (400, 4096)` `S: (400,)`
3. Top-k Basis	$V_k = [v_1, \ldots, v_k]$	`Vk = Vt[:k, :].T`	`(4096, k)` matrix
4. Latent Code	$Z_k = X_c V_k$	`Zk = Xc @ Vk`	`(400, k)` compressed
5. Reconstruction	$\hat{X}_k = Z_k V_k^T + \mu$	`X_hat = (Zk @ Vk.T) + mu`	`(400, 4096)` → `(64,64)`

5. Your 90-Minute Daily Execution Protocol

Time-Boxed Workflow

1. **Minutes 00–15 (Run & Inspect Eigenfaces):** Run `day04_pca_image_compression_denoising.py` via Antigravity. Look at Panel 2: notice how Eigenface 1 captures top-to-bottom illumination while Eigenfaces 2–5 capture eyebrows, noses, and facial contours.
2. **Minutes 15–45 (Find the 90% Elbow):** Check Panel 1 and the terminal table. Observe how just $k = 50$ components out of $4{,}096$ preserve nearly 90% of total variance—achieving an **81.9× compression ratio** per encoded image vector.
3. **Minutes 45–70 (Tune the Denoising Bottleneck):** In the denoising experiment, test `k = 5` (blurry ghost face, underfitted) vs. `k = 40` (sharp clean face, peak PSNR) vs. `k = 350` (noise returns because noisy tail eigenvectors are re-admitted!).
4. **Minutes 70–90 (Commit Visual Artifact):** Save `day04_pca_artifact.png` and commit Day 4 to your GitHub sprint repository.

> **How Day 4 Connects to Deep Autoencoders & LoRA Hybrids**
>
> Today's linear projection $Z_k = X_c V_k$ and reconstruction $\hat{X}_k = Z_k V_k^T + \mu$ is mathematically identical to a **1-layer Autoencoder without non-linear activation functions**! On **Day 18 (Convolutional Autoencoder Anomaly Detector)**, you will replace V_k and V_k^T with deep Conv2D Encoder and ConvTranspose2D Decoder blocks. Furthermore, modern LLM & Vision-Transformer fine-tuning (**LoRA: Low-Rank Adaptation**) uses the exact low-rank matrix decomposition $\Delta W = A \cdot B^T$ you built today.